PROJECT SPECIFICATION & SYSTEM DESIGN

CloudPilot

Cloud Resource Cost Intelligence & Optimization Platform

A full-stack platform for cloud cost visibility, anomaly detection, budgeting, forecasting and resource optimization recommendations.

Technology stack
Java 21 · Spring Boot · React · TypeScript · PostgreSQL · AWS

Project scope
Billing ingestion · Cost analytics · Budget alerts · Anomaly detection · Forecasting · Resource utilization reporting

PROJECT PURPOSE

Build a reliable application that ingests cloud billing data, explains spending trends, detects unusual cost changes, manages budgets, estimates future spend and presents evidence-based optimization recommendations.

Core principle: prioritize accuracy, explainability, security and human-reviewed recommendations. The initial release must not automatically stop, resize or delete cloud resources.

1. Product Overview

CloudPilot is a cloud FinOps and cost intelligence platform for engineering and finance teams. It combines billing records with available resource metrics to help users understand where money is being spent, identify unusual changes and make informed optimization decisions.

Problem statement

- Cloud costs can rise unexpectedly, and teams need to identify the services, regions or resources driving the change.
- Budget owners need timely threshold alerts and a reasonable estimate of month-end spending.
- Resource optimization requires evidence from utilization metrics, not assumptions based only on cost.
- Billing imports must remain accurate even when files contain malformed rows, overlap or are uploaded more than once.

Primary user roles

Role	Responsibilities
Administrator	Manage users, cloud account records, imports and application configuration.
Cloud / Engineering	Investigate cost drivers, anomalies and resource utilization.
Budget Owner	Create budgets, review forecasts and receive alerts.
Read-only Viewer	View dashboards, reports and recommendations without changing settings.

2. Scope

Included in the initial release

- Authentication and role-based access control.
- Billing CSV upload, validation, import status, row-level errors and import history.
- Cost summaries and trends by date, service, region, account and resource identifier when available.
- Budgets, configurable thresholds and notification history.
- Explainable rule-based cost anomaly detection.
- A baseline month-end forecast with documented assumptions and evaluation.
- Read-only AWS cost integration, subject to access and credentials being available.
- Resource utilization reports and human-reviewed optimization candidates where metrics exist.
- Automated tests, API documentation, Docker-based setup and operational documentation.

Excluded from the initial release

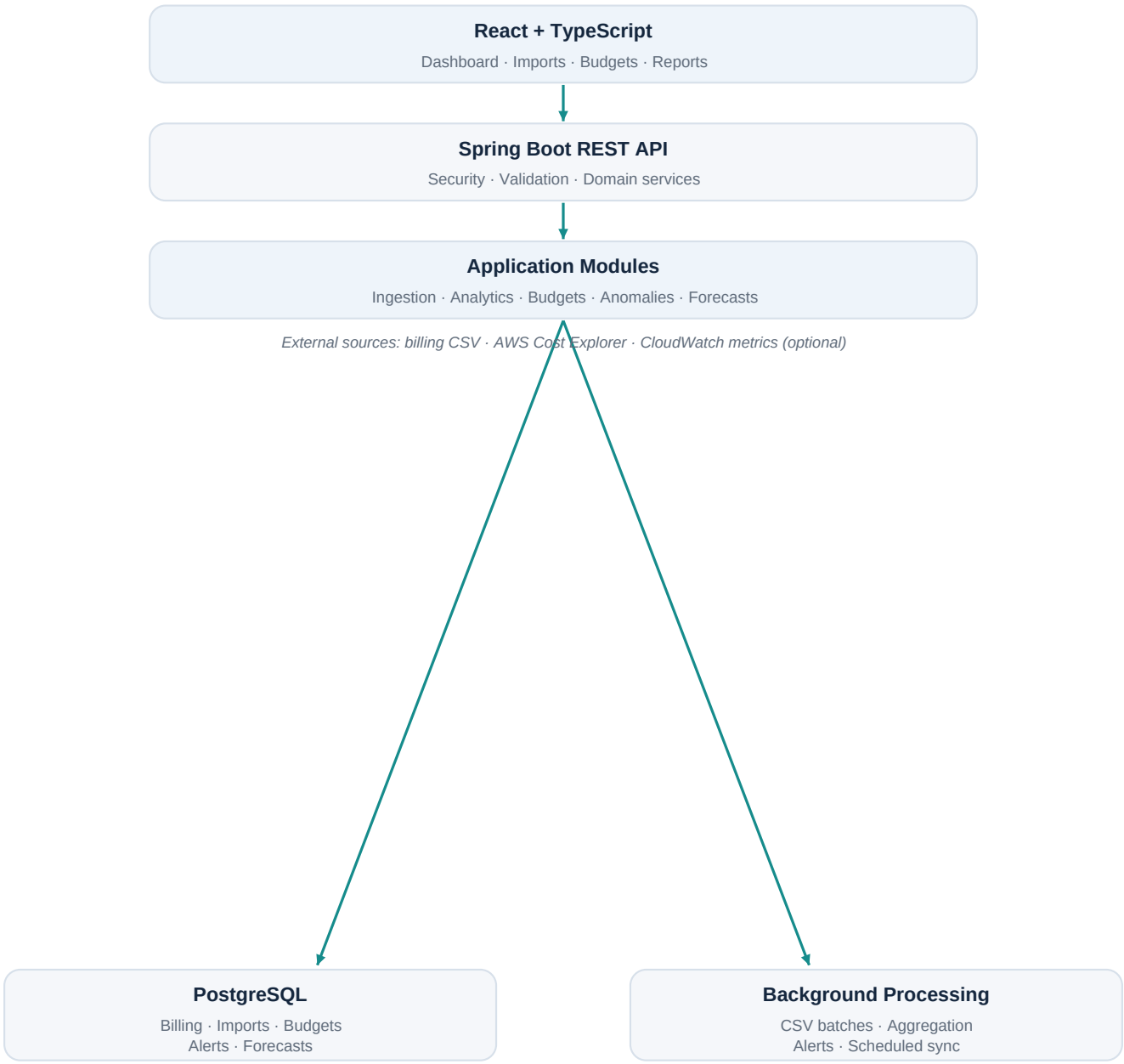
- Automatic stopping, resizing or deletion of cloud resources.
- Guaranteed savings claims or guaranteed billing forecasts.
- Advanced machine learning before a baseline model and evaluation dataset exist.
- Multi-cloud provider support beyond a clean adapter boundary unless explicitly approved.

3. Technology Stack

Layer	Technology	Purpose
Language	Java 21	Backend logic and data processing.
Backend	Spring Boot 3.x	REST APIs, validation, security and business logic.
Security	Spring Security	Authentication and authorization.
Database	PostgreSQL	Billing records, imports, budgets, alerts and reports.
Persistence	Spring Data JPA / Hibernate	Database access and persistence.
Migrations	Flyway	Versioned database schema changes.
Frontend	React + TypeScript + Vite	Interactive user interface.
UI / charts	Tailwind CSS + Recharts	Responsive layouts and visual analytics.
File parsing	Apache Commons CSV; Apache POI if XLSX is included	CSV/Excel parsing and validation.
Background work	Spring scheduling and bounded batch processing	Imports, aggregations, alerts and sync jobs.
Testing	JUnit 5, Mockito, Testcontainers, Vitest	Unit, integration and frontend tests.
Deployment	Docker Compose + GitHub Actions	Repeatable local setup and continuous integration.
AWS integration	AWS SDK for Java; Cost Explorer where available	Read-only cost data retrieval.

4. System Design

Use a modular monolith for the first release. Keep billing ingestion, analytics, budgets, anomaly detection, forecasting and provider integration in separate modules. A separate background-processing process may be used when it improves reliability, but microservices are not required for the initial version.



Core components

- **Frontend:** dashboard, billing imports, cost reports, budgets, alerts and recommendations.
- **REST API:** authentication, request validation, authorization, pagination and consistent error responses.
- **Ingestion module:** file metadata, parsing, validation, deduplication and batch persistence.
- **Analytics module:** aggregation queries, time-series responses, service breakdowns and top cost drivers.
- **Rules and forecasting modules:** budget evaluation, anomaly calculations and forecast generation.
- **AWS adapter:** read-only provider integration isolated from core domain logic.
- **PostgreSQL:** durable source records, import state, alert history and generated analytics.

Architecture principles

- Keep controllers thin and domain logic in testable service classes.
- Use DTOs instead of exposing database entities directly.
- Use pagination and server-side filtering for large result sets.
- Process large files in bounded batches instead of loading them entirely into memory.

- Keep provider-specific code behind an adapter/service boundary.
- Never commit credentials or secrets to source control.

5. Functional Requirements

5.1 Authentication and authorization

- Provide login and current-user endpoints.
- Support Administrator, Cloud/Engineering, Budget Owner and Read-only Viewer roles.
- Enforce authorization in backend endpoints; frontend visibility alone is not security.
- Never expose passwords, secret values or sensitive internal exception details.

5.2 Billing data ingestion

- Upload a supported billing CSV and create an import record with status and timestamps.
- Validate required columns, dates, numeric values, currency and malformed rows.
- Display a preview or sample of parsed rows and a summary of accepted and rejected rows.
- Use bounded batch processing and preserve row-level errors.
- Support import states such as RECEIVED, PROCESSING, COMPLETED, COMPLETED_WITH_ERRORS and FAILED.
- Prevent duplicate cost counting using a documented file-fingerprint and/or stable source-record deduplication strategy.

5.3 Cost analytics

- Display total spend for a selected period and compare it with a comparable previous period.
- Break down cost by service, region, account, usage type and resource identifier where source data provides these fields.
- Provide daily/monthly trends, top cost drivers and filterable detail tables.
- Document which cost measure is being used, such as unblended or amortized cost, when available.
- Handle missing data and currencies explicitly; do not silently combine incomparable amounts.

5.4 Budgets and alerts

- Allow budget owners to set a budget period, currency and configurable thresholds such as 50%, 80% and 100%.
- Evaluate thresholds after successful imports and on a scheduled cadence.
- Prevent duplicate notifications for the same threshold event unless repeat alerts are configured.
- Track trigger amount, threshold, evaluation time and delivery status.
- Provide in-app alert history; email delivery can be configured when an SMTP provider is available.

5.5 Anomaly detection

- Start with an explainable rule-based baseline.
- Compare recent costs with a configurable historical baseline, such as a rolling average or median.
- Use minimum absolute and percentage deviation thresholds to reduce noise.
- Store observed cost, baseline, deviation, period, severity and detection timestamp.
- Allow users to mark findings reviewed or false positive and record a note.
- Evaluate the algorithm using synthetic normal data and injected cost spikes.

5.6 Cost forecasting

- Implement a baseline forecast such as month-to-date spend plus average daily spend multiplied by remaining days.
- Display assumptions, forecast period and last-updated time.
- Evaluate against completed historical periods using an appropriate metric such as MAE or MAPE.
- Clearly label predictions as estimates, not guaranteed final bills.

5.7 Resource utilization and recommendations

- Retrieve utilization metrics only when the required source and permissions are available.
- Correlate resource identifiers carefully; not all billing records map directly to a metric resource.
- Flag possible low-utilization resources for human review and include evidence, lookback period and caveats.
- Show estimated savings only when a defensible pricing and utilization calculation exists.
- Never automatically stop, resize or delete resources.

5.8 Reporting and operations

- Export filtered cost reports as CSV.
- Provide import history, processing errors, health/readiness endpoints and operational logs.
- Use consistent error responses and server-side request validation.
- Document local setup, tests, configuration, deployment and database recovery.

6. Database Design

The following schema is a recommended starting point. Define an ER diagram, primary and foreign keys, data types, constraints and indexes before implementation. Add or adjust fields where justified and document the design decisions.

Table	Suggested fields	Purpose
users	id UUID PK; full_name; email UNIQUE; password_hash; role; is_active; created_at; updated_at	Authentication and access roles.
cloud_accounts	id UUID PK; provider; display_name; external_account_id; connection_status; created_at	Logical account record; no plain-text secrets.
billing_imports	id UUID PK; cloud_account_id FK; filename; file_fingerprint; status; total_rows; accepted_rows; rejected_rows; started_at; completed_at; error_summary	Import lifecycle and duplicate detection.
cost_records	id BIGINT/UUID PK; import_id FK; cloud_account_id FK; usage_date; service; region; resource_id NULL; usage_type; currency; cost_amount NUMERIC; source_record_key	Normalized cost line items.
budgets	id UUID PK; cloud_account_id FK; name; period_type; period_start; period_end; budget_amount NUMERIC; currency; thresholds JSONB; is_active	Budget configuration.
budget_alerts	id UUID PK; budget_id FK; threshold_percent; observed_amount; triggered_at; delivery_status; dedupe_key UNIQUE	Alert events and duplicate prevention.
cost_anomalies	id UUID PK; cloud_account_id FK; period_start; period_end; service; observed_amount; baseline_amount; deviation_percent; severity; status; detected_at; review_note	Explainable anomaly findings.
cost_forecasts	id UUID PK; cloud_account_id FK; period_start; period_end; predicted_amount; currency; method; model_version; generated_at	Forecasts and reproducibility.
resource_metrics	id BIGINT/UUID PK; cloud_account_id FK; resource_id; metric_name; metric_timestamp; metric_value; unit; source	Available utilization metrics.
optimization_recommendations	id UUID PK; cloud_account_id FK; resource_id; recommendation_type; evidence JSONB; estimated_savings NUMERIC NULL; currency NULL; status; created_at; reviewed_at	Evidence-backed recommendations.

Database requirements

- Use NUMERIC/DECIMAL for money, not floating-point values.
- Define a UTC timestamp convention and preserve the source usage date/time zone where relevant.
- A file fingerprint prevents exact duplicate-file processing, but overlapping files may need stable line-item keys as well.
- Index measured query paths, including usage_date, cloud_account_id, service and import_id as appropriate.
- Use transactions for batch persistence where practical and make import recovery explicit.
- Use Flyway migrations; avoid destructive automatic schema updates in deployed environments.

7. REST API Specification

Base path: `/api/v1`. Document all endpoints in OpenAPI/Swagger, including request/response schemas, validation, access permissions and error cases.

Method	Endpoint	Purpose
POST	/auth/login	Authenticate a user.
GET	/auth/me	Return current user and roles.
GET	/cloud-accounts	List permitted cloud accounts.
POST	/cloud-accounts	Register a logical cloud account.
POST	/billing-imports	Upload a billing CSV and create an import.
GET	/billing-imports	List imports with pagination and filters.
GET	/billing-imports/{id}	Get import status and row counts.
GET	/billing-imports/{id}/errors	Inspect import validation errors.
GET	/costs/summary	Return total spend for a period.
GET	/costs/trends	Return daily/monthly trend series.
GET	/costs/breakdown	Group cost by service, region or another dimension.
GET	/costs/top-drivers	Return largest cost contributors or increases.
GET	/budgets	List budgets.
POST	/budgets	Create a budget.
PATCH	/budgets/{id}	Update a budget.
GET	/alerts	List budget alerts and anomaly findings.
GET	/anomalies	Filter anomalies by date, service and status.
POST	/anomalies/{id}/review	Review a finding and record a note.
GET	/forecasts	Return forecasts and generation metadata.
GET	/resources/utilization	Return available utilization metrics.
GET	/recommendations	List optimization candidates.
PATCH	/recommendations/{id}/review	Record a human review decision.
GET	/reports/costs/export	Export a filtered cost report as CSV.
GET	/health	Basic application health.
GET	/ready	Readiness check for required dependencies.

API standards

- Use pagination and enforce a maximum page size.
- Use consistent HTTP status codes and a centralized error response format.
- Validate date ranges, currencies, amounts, file sizes and content types.
- Never expose password hashes, cloud secrets or internal stack traces.
- Enforce object-level authorization on every endpoint.
- Use idempotency or stable identifiers for operations that may be retried.

8. Frontend Screens

Screen	Expected functionality
Overview dashboard	Total cost, month-over-month change, daily trends, top services, budget status and recent alerts.
Billing imports	Upload, preview, processing status, accepted/rejected row counts and downloadable error report.
Cost explorer	Date range, account, service and region filters; trend charts and detail table.
Budget management	Create/edit budgets, thresholds, progress and alert history.
Anomaly centre	Observed versus baseline values, deviation, severity, review status and notes.
Forecast view	Estimated month-end spend, assumptions, forecast period and historical error.
Resource optimization	Available metrics, flagged candidates, evidence, caveats and review status.
Settings	User roles, account records and application configuration as permitted.

9. Twelve-Week Implementation Plan

Week 1 — Requirements and design

Tasks: Review sample billing formats; define user stories and acceptance criteria; design architecture and ERD; create repository and local environment.

Deliverable: Requirements, ERD, architecture and runnable skeleton.

Week 2 — Application foundation

Tasks: Implement authentication, roles, DTO validation, exception handling, database migrations and protected React routes.

Deliverable: Secure application foundation.

Week 3 — Billing ingestion

Tasks: Implement CSV upload, import record, bounded parsing, column validation, row-level errors and import status UI.

Deliverable: End-to-end CSV import.

Week 4 — Normalization and summaries

Tasks: Persist normalized records, implement deduplication and cost summary APIs; reconcile results against sample calculations.

Deliverable: Initial cost dashboard and reconciliation evidence.

Week 5 — Analytics

Tasks: Build daily/monthly trends, top cost drivers, filters, pagination and CSV export; measure and improve slow queries.

Deliverable: Interactive cost explorer.

Week 6 — Budgets and alerts

Tasks: Implement budgets, threshold evaluation, alert history and duplicate-notification prevention.

Deliverable: Working budget workflow.

Week 7 — Anomaly detection

Tasks: Build a rule-based baseline, deviation thresholds, persisted findings, review workflow and synthetic test cases.

Deliverable: Explainable anomaly centre.

Week 8 — Forecasting

Tasks: Implement baseline forecast, document assumptions and evaluate historical forecast error.

Deliverable: Forecast view and evaluation report.

Week 9 — AWS cost integration

Tasks: Add a read-only AWS cost adapter if access is available; handle credentials, pagination, rate limits, errors and data freshness.

Deliverable: AWS sync path or documented mock adapter.

Week 10 — Utilization and recommendations

Tasks: Use available resource metrics, correlate identifiers and create conservative evidence-backed optimization candidates.

Deliverable: Utilization report and reviewable recommendations.

Week 11 — Quality and performance

Tasks: Add integration/edge-case tests, large-file tests, query profiling, Docker Compose, CI and security review.

Deliverable: Release candidate and test/benchmark report.

Week 12 — Release and handover

Tasks: Deploy to approved environment, run acceptance tests, fix critical defects and complete setup/API/operations documentation.

Deliverable: Deployed MVP and final handover.

10. Testing and Acceptance Criteria

The project is accepted based on correct behavior and evidence, not only on visual completeness. Tests must cover normal workflows, malformed data, duplicate operations, edge cases and authorization failures.

Required acceptance scenarios

- Upload a valid sample CSV and verify import status, row counts and cost totals.
- Upload a malformed file and confirm actionable validation errors without exposing stack traces.
- Re-upload the same file and verify that the deduplication strategy prevents accidental double counting.
- Reconcile daily and service-level aggregates against independently calculated expected results.
- Create a budget and verify that configured thresholds trigger correctly without duplicate alert events.
- Inject a synthetic cost spike and verify that anomaly findings include baseline and deviation.
- Generate a forecast and show its assumptions and historical evaluation metric.
- Attempt unauthorized access to another user's restricted account/report and verify the backend blocks it.
- Run a large-file test and record file size, processing duration, observed memory behavior and error counts.
- Run the full automated test suite and start the application from a clean Docker setup.

Test levels

Level	Expected coverage
Unit	Parsing, validation, aggregation, budget thresholds, anomaly calculations and forecast logic.
Integration	Repositories, migrations, transaction behavior, import lifecycle and API authorization.
Frontend	Forms, validation errors, loading/empty states, filters and key workflows.
End-to-end	CSV import through dashboard, budget alerts, anomaly review and deployment smoke tests.
Performance	Large-file processing and aggregation queries with documented test environment and results.

11. Evaluation Criteria (100 points)

Category	Points	Evaluation focus
Backend architecture and code quality	15	Module boundaries, DTOs, service design, validation and maintainability.
Database and data correctness	15	Constraints, deduplication, monetary precision, reconciliation and query design.
Ingestion reliability	15	Bounded-memory processing, row-level errors, recoverable state and safe re-import.
Analytics and anomaly detection	15	Correct aggregations, explainable baselines, tests and limitations.
React frontend	10	Dashboard usability, charts, filters, forms and error states.
Testing and security	15	Automated tests, authorization, validation and safe secrets handling.
Deployment and documentation	5	Docker, CI and reproducible setup.
Ownership and problem-solving	10	Independent debugging, clear trade-offs and working demonstrations.

Critical security or data-integrity defects must be resolved before the application is considered ready for release.

12. Engineering and Operational Standards

Code quality

- Use meaningful commits, feature branches and pull requests.
- Keep controllers thin and business rules in testable services.
- Use structured logs with useful import/request identifiers; never log secrets.
- Maintain clear naming, consistent formatting and documented design decisions.

Security and cloud safety

- Use synthetic or sanitized billing samples during development.
- Use read-only AWS permissions and least-privilege access.
- Store credentials in environment configuration or an approved secret manager, never in source control.
- Do not execute stop, resize or delete operations against cloud resources.
- Label forecasts and savings estimates clearly and include supporting evidence.

Operational expectations

- Provide health/readiness endpoints and meaningful logs.
- Document configuration, local startup, tests, deployment and database backup/restore.
- Record known limitations, data freshness and failed import details.
- Keep data processing recoverable and prevent repeated runs from silently duplicating records.

13. Final Deliverables

- Source repository with understandable commit history.
- System architecture diagram and database ER diagram.
- Database migrations and synthetic sample billing data.
- OpenAPI/Swagger documentation and a Postman collection.
- Automated test suite and test instructions.
- Docker Compose setup and CI workflow.
- README with prerequisites, configuration, run/test/deploy instructions and known limitations.
- Screenshots or a short demonstration of the main workflows.
- Known-issues list and proposed future enhancements.
- Technical presentation explaining architecture, important trade-offs, failure cases and measured results.

14. Future Enhancements

- Additional cloud-provider adapters.
- More advanced forecast methods evaluated against the baseline.
- Improved anomaly models with explainability and alert tuning.
- Scheduled incremental billing synchronization.
- Tag-based cost allocation and team/department chargeback.
- Additional notification integrations and advanced observability.

Definition of done

A feature is complete when its business behavior is implemented, inputs are validated, permissions are enforced, relevant automated tests pass, API/UI behavior is documented and the feature can be demonstrated in a repeatable environment.

End of Project Specification & System Design